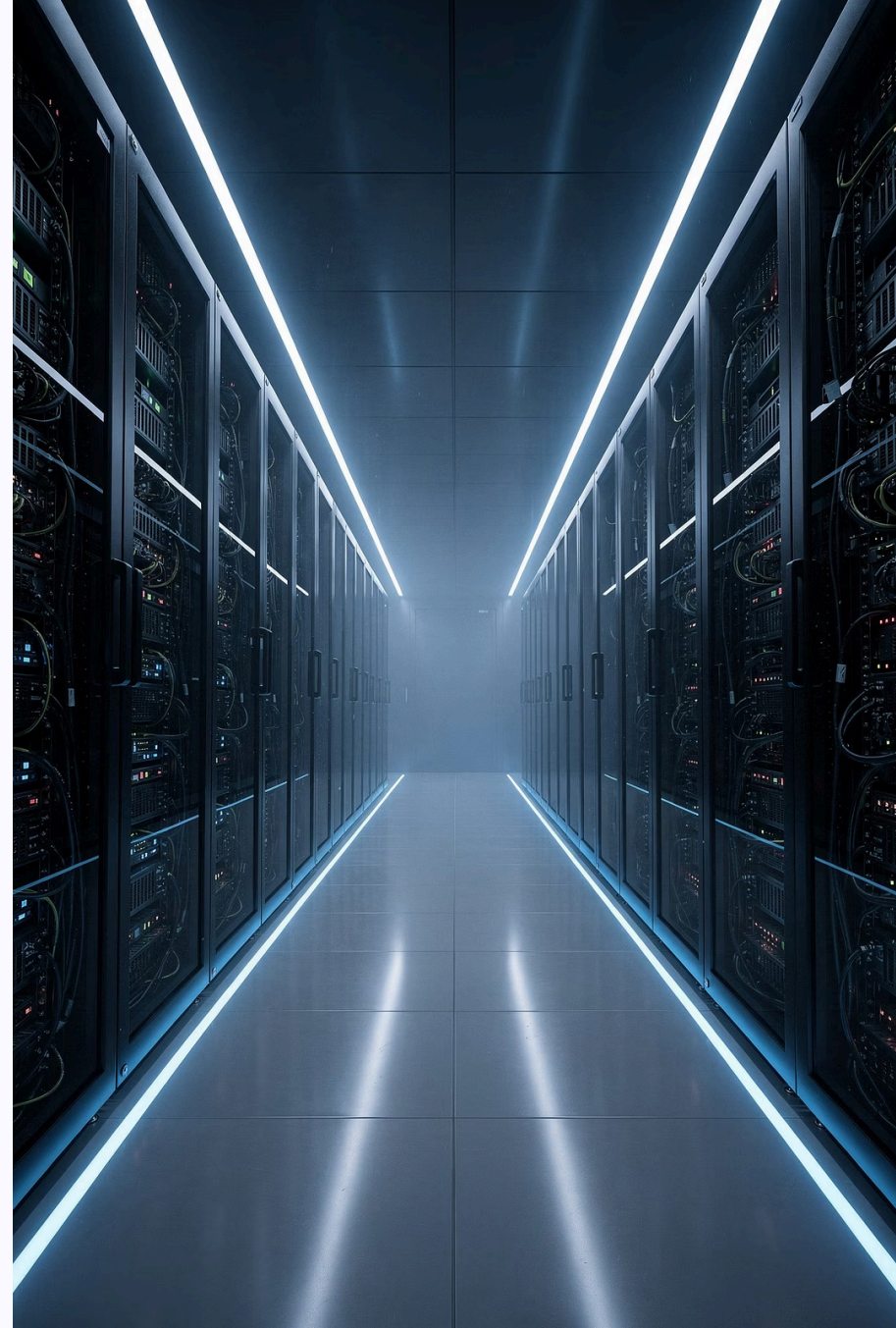


API Gateway en Spring Boot 3

Implementación práctica con WebMVC para arquitecturas de microservicios legacy
– desde la teoría hasta las pruebas en entorno local.

ARQUITECTURA DE MICROSERVICIOS

SPRING BOOT 3 · WEBMVC · JDK 21





¿Por qué un Gateway Independiente?

El reto principal en entornos reales es la **coexistencia de tecnologías heterogéneas**. Un gateway independiente actúa como mediador neutral.



Capa Moderna

Spring Boot 3.x + JDK 21

Virtual Threads, última generación de Spring Cloud Gateway, rendimiento optimizado y soporte LTS extendido.



Servicios Legacy

Spring 4 + JDK 8/11

Microservicios en producción con años de madurez. Migrarlos implica riesgo, coste y tiempo que no siempre se justifica.



El Gateway como Puente

Coexistencia sin ruptura

El gateway enruta hacia cualquier servicio independientemente de su stack tecnológico, preservando el ecosistema existente.

Flujo de una Petición: Norte-Sur vs. Este-Oeste



Tráfico Norte-Sur (externo)

Todo cliente externo – Postman, navegador o aplicación móvil – **solo conoce el puerto 8080** del gateway. Nunca accede directamente a los microservicios internos.

Tráfico Este-Oeste (interno)

La comunicación **entre microservicios** puede ocurrir directamente (service-to-service) o también enrutarse a través del gateway para aplicar políticas centralizadas de seguridad y trazabilidad.

Paso 1 – Creación de mc-saludo

El microservicio mc-saludo representa un **servicio legacy** basado en Spring 4. Es el destino final al que el gateway enrutará las peticiones. Se desplegará en el puerto **8081**.

Configuración del entorno

Framework: Spring 4.x (Spring MVC)

Puerto: 8081

Contexto: /api/v1/saludo

Empaquetado: WAR o JAR según el servidor de aplicaciones legacy.

application.properties

```
server.port=8081
spring.application.name=mc-saludo
```

Dependencia Maven mínima

```
<dependency>
  <groupId>
    org.springframework.boot
  </groupId>
  <artifactId>
    spring-boot-starter-web
  </artifactId>
</dependency>
```

Código del Controlador: SaludoController

La clase controladora es la pieza central del microservicio. Utiliza las anotaciones estándar de Spring MVC para mapear rutas HTTP a métodos Java de forma declarativa.

```
package com.example.mcsaludo.controller;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
@RequestMapping("/api/v1/saludo")
public class SaludoController {

    @GetMapping
    public String holaMundo() {
        return "hola mundo";
    }
}
```

@RestController

Combina `@Controller` y `@ResponseBody`. Todos los métodos serializan la respuesta directamente al cuerpo HTTP.

@RequestMapping

Define el prefijo base de la ruta para todos los endpoints del controlador: `/api/v1/saludo`.

@GetMapping

Mapea las peticiones HTTP GET sin ruta adicional al método `holaMundo()`.

Endpoint Temprano: Verificación Inicial

¿Por qué probar antes del gateway?

Antes de añadir capas de infraestructura, es fundamental verificar que el microservicio `mc-saludo` funciona de forma **autónoma e independiente**. Esto nos permite aislar problemas: si el gateway falla, sabemos que el error no está en el servicio origen.

URL de prueba directa

```
GET http://localhost:8081/api/v1/saludo
```

Respuesta esperada

```
HTTP/1.1 200 OK
Content-Type: text/plain

hola mundo
```

Flujo de verificación

01

Arrancar mc-saludo

Ejecutar `mvn spring-boot:run` y confirmar que el puerto 8081 está activo en los logs de inicio.

02

Abrir Postman o curl

Lanzar una petición GET directa al puerto 8081, sin pasar por ningún intermediario.

03

Validar respuesta 200

Confirmar que el cuerpo contiene exactamente `"hola mundo"` y el status code es `200 OK`.

Paso 2 – Creación de mc-api-gateway

El nuevo proyecto del gateway se genera desde **Spring Initializr** (start.spring.io) con la stack tecnológica más moderna. Es un proyecto independiente con su propio ciclo de vida.

Configuración en Spring Initializr

Project: Maven

Language: Java

Spring Boot: 3.x (última estable)

Java: 21 (LTS)

Artifact: mc-api-gateway

Packaging: JAR

Dependencias seleccionadas

Gateway (MVC)

spring-cloud-starter-
gateway-server-webmvc –
versión Servlet, basada en
Tomcat.

Spring Web

spring-boot-starter-web –
soporte WebMVC completo
y servidor embebido
Tomcat.

 El puerto del gateway se configurará en 8080 – la convención HTTP estándar para puntos de entrada.

El Dilema de Arquitectura: Reactivo vs. Sincrónico

Spring Cloud Gateway ofrece **dos sabores arquitectónicos** con implicaciones técnicas significativas. La elección correcta depende del stack existente y los requisitos de rendimiento.

ENFOQUE REACTIVO (NETTY)



- ☒ PROJECT REACTOR
- ☒ WEBFLUX
- ☒ NO BLOQUEANTE
- ☒ I/O ASÍNCRONO
- ☒ ALTA CONCURRENCIA
- ☒ INCOMPATIBLE SERVLET API

ENFOQUE SINCRÓNICO (TOMCAT WEBMVC)



- ☒ SERVLET API
- ☒ SPRING MVC
- ☒ BLOQUEANTE
- ☒ COMPATIBLE FILTROS LEGACY
- ☒ FÁCIL MIGRACIÓN
- ☒ MODELO HILOS TRADICIONAL

⚠ Regla de oro: Si tu ecosistema usa filtros Servlet, Spring Security tradicional o librerías bloqueantes, **elige siempre la variante WebMVC**. Mezclar WebFlux con código bloqueante genera errores difíciles de diagnosticar.

Dependencia Clave en pom.xml

La dependencia `spring-cloud-starter-gateway-server-webmvc` es la pieza central que habilita el motor de enrutamiento basado en **Servlet API**, diferenciándola de la versión reactiva estándar.

```
<!-- Spring Cloud Gateway - Variante WebMVC (Servlet/Tomcat) -->
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>
    spring-cloud-starter-gateway-server-webmvc
  </artifactId>
</dependency>

<!-- BOM de Spring Cloud (gestión de versiones) -->
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2023.0.x</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

❏ **⚠ Importante:** No incluir `spring-cloud-starter-gateway` (sin el sufijo `-server-webmvc`), ya que ese artefacto tira de WebFlux/Netty y causará conflictos de dependencias con Tomcat.

Clase Principal: ApiGatewayApplication

La clase de entrada del gateway es deliberadamente **minimalista**. Toda la inteligencia de enrutamiento reside en la configuración externa (`application.yml`), no en el código Java.

```
package com.example.mcapigateway;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication

public class ApiGatewayApplication {

    public static void main(String[] args) {
        SpringApplication.run(
            ApiGatewayApplication.class, args
        );
    }
}
```

Principio de diseño

@SpringBootApplication

Anotación compuesta que activa auto-configuración, escaneo de componentes e importación de configuración. Spring Cloud Gateway se auto-configura al detectar la dependencia en el classpath.

Código = Cero lógica de negocio

El gateway no debe contener lógica de enrutamiento en Java. Esto garantiza que las rutas se puedan modificar **sin recompilar ni redeployar** el artefacto.

Configuración externalizada

Siguiendo el principio **12-Factor App**, toda configuración dinámica se externaliza en `application.yml` o variables de entorno.

Paso 3 – El Motor de Reglas: application.yml

El fichero `application.yml` es el **cerebro del gateway**. Define qué peticiones acepta, hacia dónde las envía y qué transformaciones aplica. Un YAML mal indentado puede deshabilitar rutas enteras sin generar errores evidentes.

Configuración base del servidor

```
server:  
  port: 8080  
  
spring:  
  application:  
    name: mc-api-gateway
```

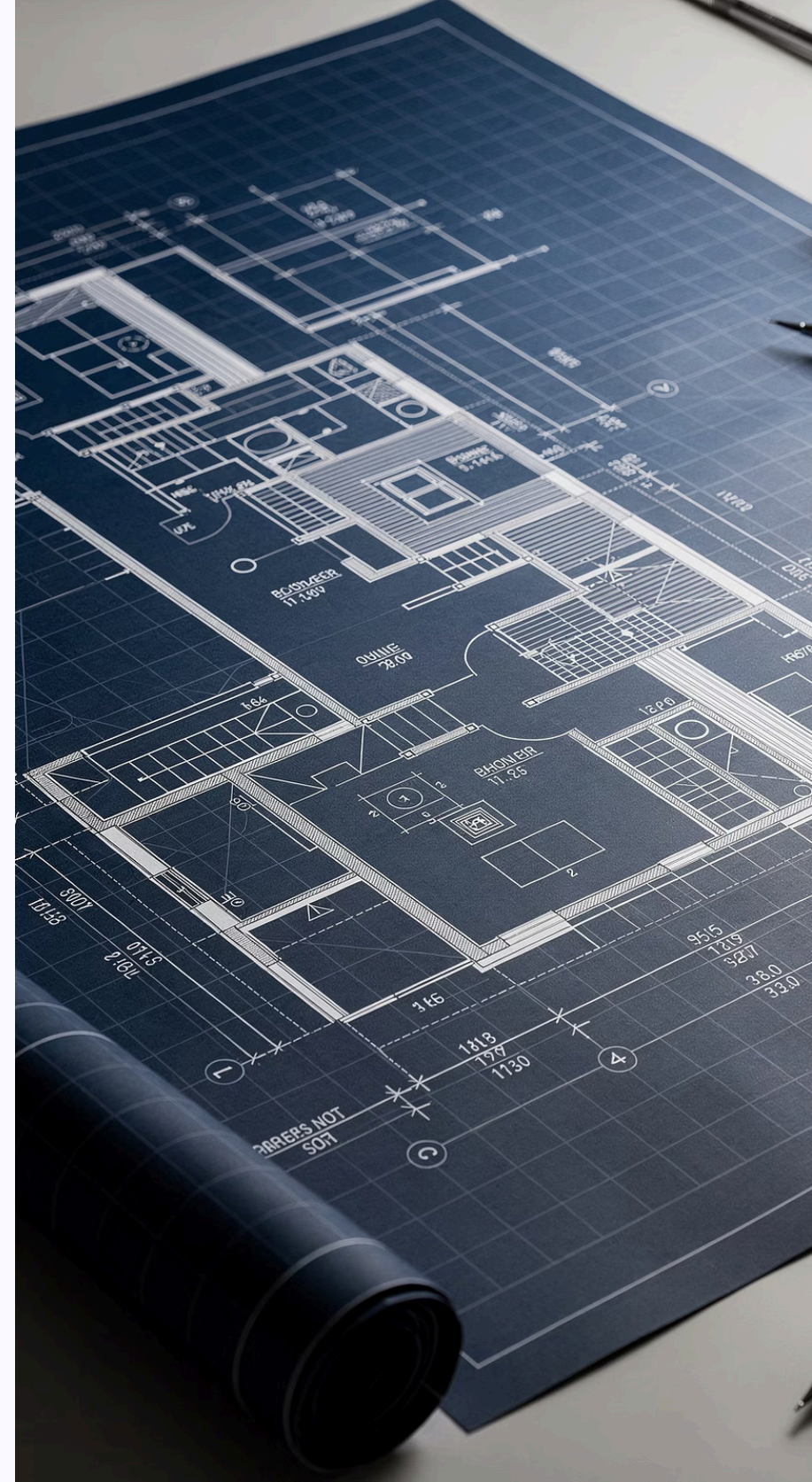
¿Por qué YAML y no Properties?

Jerarquía visual

YAML representa la anidación de propiedades de forma natural, crucial para estructuras complejas como rutas del gateway.

Menos repetición

Evita prefijos largos repetidos (`spring.cloud.gateway.mvc.routes[0].id`) que hacen el `.properties` ilegible.



Código YAML del Gateway — Parte 1: Estructura Base

El prefijo `spring.cloud.gateway.mvc.routes` es **obligatorio y sensible a la variante**. El segmento `mvc` distingue esta configuración del gateway reactivo (`spring.cloud.gateway.routes`).

```
spring:
  cloud:
    gateway:
      mvc:      # ← Clave diferenciadora: variante WebMVC
      routes:   # ← Lista de reglas de enrutamiento
        - id: mc-saludo      # Identificador único
          uri: http://localhost:8081    # Destino del tráfico
          predicates:           # Condiciones de activación
            - Path=/api/v1/saludo
            - Path=/api/v1/saludo/**
```

 **Error frecuente:** Usar el prefijo `spring.cloud.gateway.routes` (sin `mvc`) en un proyecto WebMVC resulta en que las rutas son ignoradas silenciosamente. El gateway arranca pero no enruta ninguna petición.

Código YAML del Gateway: Detalle de Ruta

Anatomía de una ruta

```
spring:
  cloud:
    gateway:
      mvc:      # ← Clave diferenciadora: variante WebMVC
      routes:   # ← Lista de reglas de enrutamiento
        - id: mc-saludo      # Identificador único
          uri: http://localhost:8081    # Destino del tráfico
          predicates:           # Condiciones de activación
            - Path=/api/v1/saludo
            - Path=/api/v1/saludo/**
```

Desglose de cada propiedad

1**id: mc-saludo**

Identificador único de la ruta. Aparece en los logs de trazas para identificar qué regla se activó. Debe ser descriptivo y único en todo el fichero.

2**uri: http://localhost:8081**

URL base del microservicio destino. En producción, esta URI apuntaría a un nombre DNS interno o a una URL de Kubernetes Service.

3**predicates: Path=...**

Condición que debe cumplir la petición entrante. Se definen **dos predicados**: con y sin barra final, para capturar ambas variantes de la URL correctamente.

Paso 4 – Orquestación de Arranque Local

El orden de arranque importa. Aunque el gateway no falla si el microservicio destino no está disponible al inicio, las primeras peticiones devolverán **503 Service Unavailable** hasta que el servicio esté operativo.



✅ **Tip de productividad:** Usa dos terminales o pestañas separadas en tu IDE – una por proyecto. Así puedes monitorizar los logs de ambos servicios simultáneamente durante las pruebas.



Prueba de Enrutamiento Exitoso con Postman

La llamada clave

El cliente apunta **exclusivamente al gateway** (puerto 8080). Nunca al microservicio directamente. Esto simula el comportamiento real de producción.

```
GET http://localhost:8080
/api/v1/saludo
```

```
Headers:
Accept: application/json
```

```
-- Respuesta esperada --
HTTP/1.1 200 OK
Content-Type: text/plain
```

```
hola mundo
```

Checklist de validación

1

Status 200 OK

Confirma que la ruta se activó y el microservicio destino respondió correctamente.

2

Body: "hola mundo"

La respuesta del microservicio llegó íntegra a través del gateway sin modificaciones.

3

Sin errores 502/503

Descarta problemas de conectividad entre gateway y microservicio (Bad Gateway o servicio no disponible).

Análisis de Trazas: Cómo el Gateway Intercepta el Tráfico

Con el nivel `TRACE` activado, los logs revelan el ciclo de vida completo de cada petición – desde la recepción en el gateway hasta la respuesta del microservicio destino.

```
-- Log del API Gateway (puerto 8080) --  
TRACE RoutePredicateHandlerMapping : Checking route:  
    id=mc-saludo, predicates=[Path=/api/v1/saludo]  
TRACE RoutePredicateHandlerMapping : Route matched: mc-saludo  
DEBUG DispatcherServlet : GET "/api/v1/saludo"  
DEBUG ProxyExchange : Forwarding to http://localhost:8081/api/v1/saludo  
  
-- Log de mc-saludo (puerto 8081) --  
DEBUG DispatcherServlet : GET "/api/v1/saludo"  
DEBUG SaludoController : holaMundo() invoked  
DEBUG DispatcherServlet : Completed 200 OK
```



Evaluación de Predicados

El gateway compara la URL entrante con todos los predicados definidos. El primero que coincide gana.



Reenvío Transparente

`ProxyExchange` reenvía la petición conservando headers, método HTTP y body. El cliente es ajeno al cambio de puerto.



Respuesta de Retorno

La respuesta del microservicio viaja de vuelta a través del gateway al cliente original, completando el ciclo.

Conclusión y Buenas Prácticas

Has implementado un **API Gateway funcional con Spring Boot 3 y WebMVC** que enruta tráfico hacia un microservicio legacy. Este patrón escala hacia ecosistemas de decenas de servicios con mínimos cambios de configuración.



Seguridad Centralizada

Implementa autenticación JWT, OAuth2 o API Keys en un único punto. Ningún microservicio necesita gestionar autenticación individualmente.



Gestión de CORS

Configura las políticas Cross-Origin una sola vez en el gateway. Elimina la duplicación de configuración CORS en cada microservicio del ecosistema.



Escalabilidad Futura

Añadir un nuevo microservicio es tan simple como agregar una nueva entrada en `application.yml`. Sin recompilación, sin redeployment del gateway.



Filtros Transversales

Aplica rate limiting, circuit breaker, logging de trazas y transformación de headers como filtros reutilizables en todas las rutas.

✅ **Próximo paso recomendado:** Registra el gateway en **Eureka Service Discovery** para reemplazar las URIs estáticas (`localhost:8081`) por nombres de servicio dinámicos, eliminando la dependencia de IPs y puertos hardcoded.